R4.01 Architecture logicielle

CM2 : MongoDB / Test

MongoDB

- MongoDB (de l'anglais humongous qui peut être traduit par « énorme ») est un système de gestion de base de données orienté documents, répartissable sur un nombre quelconque d'ordinateurs et ne nécessitant pas de schéma prédéfini des données (wikipedia).

- MongoDB permet de manipuler des objets structurés au format BSON (JSON binaire), sans schéma prédéterminé.

MongoDB vs Relationnel

MongoDB

- Des documents dans des collections


```
[
{ "_id": ObjectId("4efa8d2b7d284dad101e4bc7"),
"Nom": "DUMOND",
"Prénom": "Jean",
"Âge": 43 },
{ "_id": ObjectId("4efa8d2b7d284dad101e4bc8"),
"Nom": "PELLERIN",
"Prénom": "Franck",
"Adresse": "1 chemin des Loges",
"Ville": "VERSAILLES" },
]
```


SQL

- Des enregistrements dans des tables

ID	Nom	Prénom	Âge
6	DUMOND	Jean	43
7	PELLERIN	Franck	*NULL*
8	MARTIN	Emile	62
9	KING	*NULL*	51

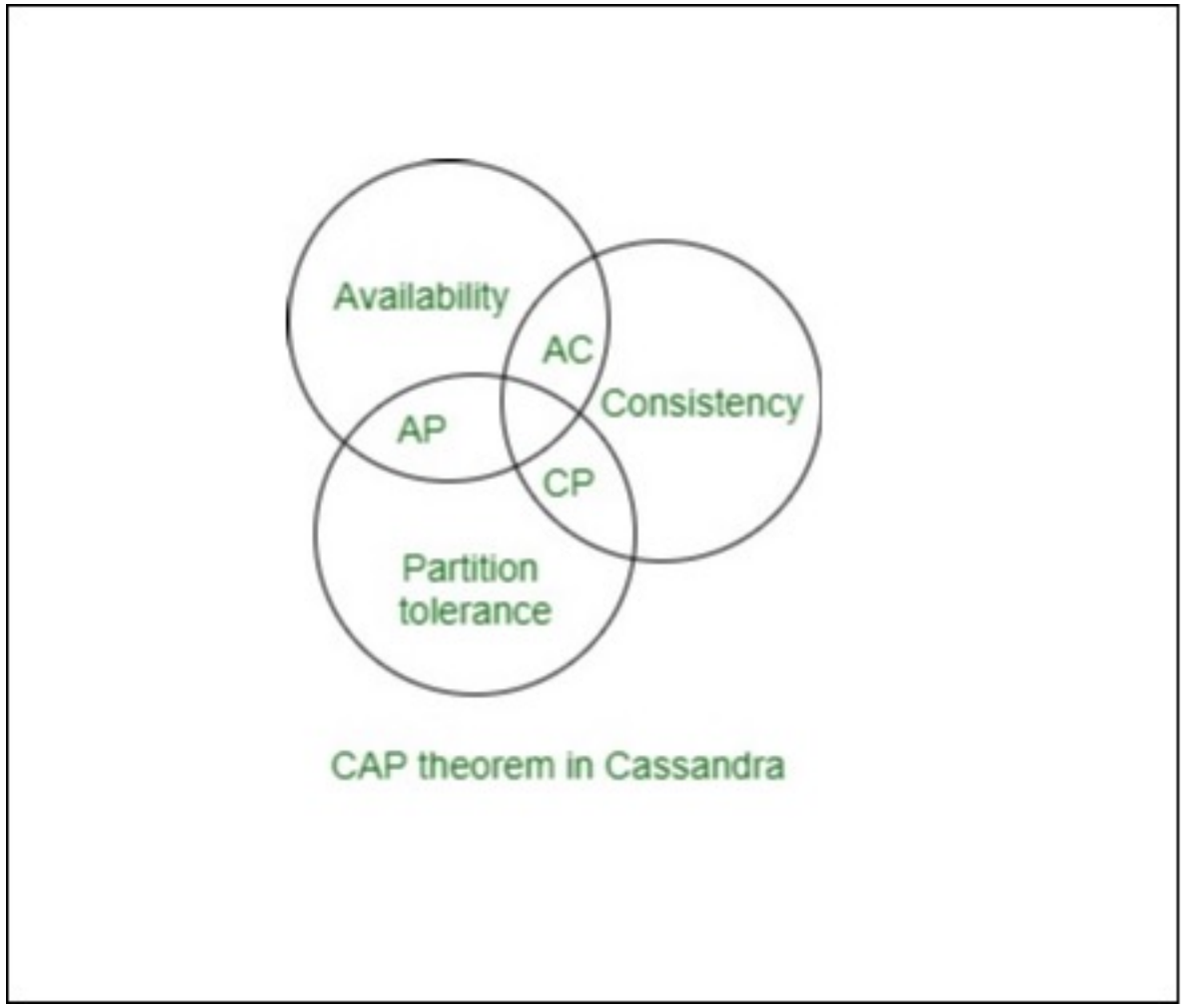
MongoDB

Pas de schéma prédéfini : Dans un document, des champs peuvent être ajoutés, supprimés, modifiés et renommés à tout moment.


```
{
 "_id": ObjectId("4efa8d2b7d284dad101e4bc7"),
"Nom": "PELLERIN",
"Prénom": "Franck",
"Âge": 29,
"Adresse": {
        "Rue" : "1 chemin des Loges",
        "Ville": "VERSAILLES" }
 }
```

MongoDB

- BD respectant CP
- BSON : *Binary JSON*
- Formats spéciaux *:* ObjectID, ISODate



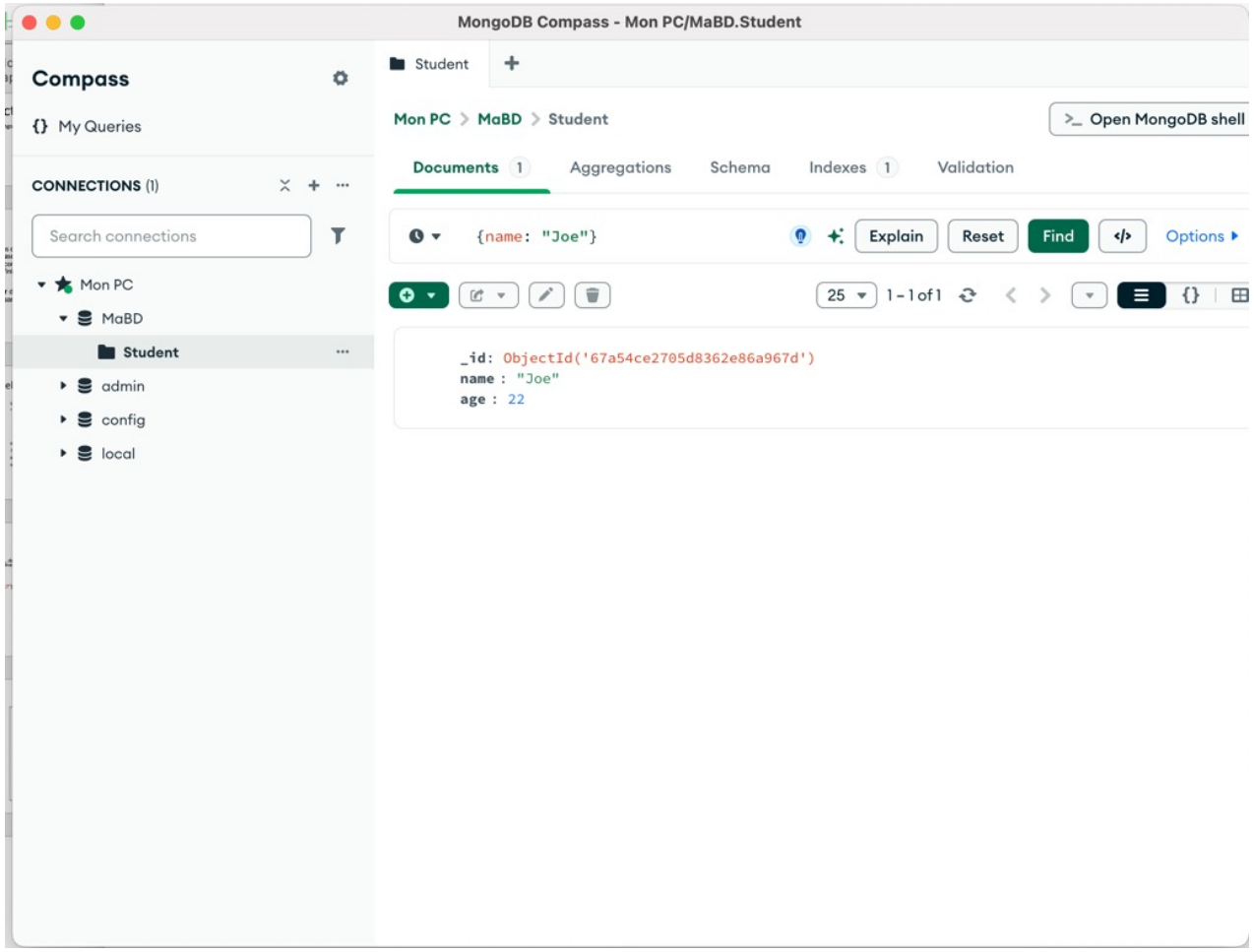
CAP theorem in Cassandra

MongoDB – shell - GUI

mongosh

- show dbs
- use une_bd
- show collections
- db.collection.find({})

…



MongoDB CRUD

- db.*collectionName*.find(args) et db.*collectionName*.findOne(args)
  - pour l'accès aux documents de la collection *collectionName*.

- db.*collectionName*.insertOne(args)
  - pour l'ajout d'un nouveau document à la collection. (insertMany existe également)
- db.*collectionName*.updateMany(args)
  - pour la mise à jour de documents. (updateOne})} existe également)
- db.*collectionName*.deleteOne(args) et db.*collectionName*.deleteMany(args)
  - pour supprimer un ou plusieurs documents de la collection.

MongoDB Schema

```
const options = {
   validator: {
      $jsonSchema : {
         bsonType: "object",
         required: ['number','name','address','position',
            'banking','bonus','status', 'contract_name',
            'bike_stands','available_bike_stands','available_bikes',
            'last_update'],
         properties : {
            number: {
               bsonType: "int",
               description: "identifiant"
            },
            name: {
               bsonType: "string",
               description: "nom"
            },
```

...

MongoDB en JS

```
'use strict'

import { MongoClient } from 'mongodb'

//connexion
const  url = "mongodb://localhost:27017"
const client = new MongoClient(url)

 //acces à la BD
    const maBD = client.db("maBD")
  //acces à la collection
    const maCollection=
maBD.collection("maCollection")
```

- Avec un serveur en mémoire

```
let mongod = await
MongoMemoryServer.create();
const uri = mongod.getUri();
const client = new MongoClient(uri)
const maBD = client.db("maBD")


const maCollection=
maBD.collection("maCollection",options)
```

MongoDB

- Exemple de requête

```
const res = await collection.findOne({number: id}, {
    projection: {_id:0},
})
```

Mongoose

Mongoose est une bibliothèque de modélisation de données d'objet (ODM) pour MongoDB et Node. js.

Populaire, permet notamment de conserver la connexion entre les modules

https://mongoosejs.com/docs/

Mongoose

- Schema pour définir le type des données
- Model permet de manipuler des documents conforme au schema

```
const positionSchema = new mongoose.Schema(
    {lat: Number,lon: Number})
const monSchema = new mongoose.Schema({
    number: Int32,
    name: String,
    address: String,

....
const StationModel = mongoose.model('Station', monSchema);
```

Moongose CRUD

Create

doc.name = 'foo’;

await doc.save();

User.collection.insert([<document 1> , <document 2>, ...], { writeConcern: <document>, ordered: <boolean> }) User.collection.insertMany([<document 1> , <document 2>, ...], { writeConcern: <document>, ordered: <boolean> })

User.collection.insert([<document 1> , <document 2>, ...], { writeConcern: <document>, ordered: <boolean> }) User.collection.insertMany([<document 1> , <document 2>, ...], { writeConcern: <document>, ordered: <boolean> })

User.create([Newuser,Newuser1,Newuser2]) .then((data)=>{ resolve(data); }).catch((err)=>{ reject(err); })

Moongose CRUD

Model.deleteMany()

Model.deleteOne()

Model.find()

Model.findById()

Model.findByIdAndDelete()

Model.findByIdAndRemove()

Model.findByIdAndUpdate()

Model.findOne()

Model.findOneAndDelete()

Model.findOneAndReplace()

Model.findOneAndUpdate()

Model.replaceOne()

Model.updateMany()

Model.updateOne()

Moongose

```
const adapter = (mongooseStation)=>{
    if (mongooseStation == null)
        return null
    mongooseStation = mongooseStation.toJSON()
    mongooseStation.last_update =
mongooseStation.last_update.getTime()
    return new Station(mongooseStation)
}
```